

Johannes Klar

Cypher Injection in Neo4j: Securing Dynamic Queries

BACHELORARBEIT

zur Erlangung des akademischen Grades
Bachelor of Science (BSc)

BACHELORSTUDIUM
Wirtschaftsinformatik

Universität Klagenfurt

BETREUUNG

Assoc.Prof. Mag. Dr. Christian Kop
Universität Klagenfurt
Institut für Artificial Intelligence und Cybersecurity

Klagenfurt, 29th July 2026

Affidavit

I hereby declare in lieu of an oath that

- the submitted academic paper is entirely my own work and that no auxiliary materials have been used other than those indicated,
- I have fully disclosed all assistance received from third parties during the process of writing the thesis, including any significant advice from supervisors,
- any contents taken from the works of third parties or my own works that have been included either literally or in spirit have been appropriately marked and the respective source of the information has been clearly identified with precise bibliographical references (e.g. in footnotes),
- I have fully and truthfully declared the use of generative models (Artificial Intelligence, e.g. ChatGPT, Grammarly Go, Midjourney) including the product version,
- to date, I have not submitted this paper to an examining authority either in Austria or abroad and that
- when passing on copies of the academic thesis (e.g. in printed or digital form), I will ensure that each copy is fully consistent with the submitted digital version.

I am aware that a declaration contrary to the facts will have legal consequences.

Johannes Klar

Klagenfurt, 29th July 2026

Abstract

Injection attacks have been a long-standing and omnipresent threat to database systems ever since their inception. While most often targeting traditional relational databases and SQL, the attack paradigm of injection has over time been successfully adapted to also take aim at NoSQL databases and their query languages. This thesis investigates the topic of Cypher Injection, that is, the concept of classic SQL injection adjusted to target the Neo4j graph database and its associated Cypher language. Existing SQL Injection Attack (SQLIA) taxonomies will be examined to assess the feasibility of transferring various types of SQLIAs to Neo4j and Cypher, as well as to identify the adaptations required. For this purpose, this thesis will more closely examine a number of frequently used yet loosely defined terms related to the construction and execution of database queries, such as *static*, *dynamic*, and *parameterized*. Dynamic queries in particular will be given special attention, as they make up the most problematic class of queries with regard to injection attacks. Consequently, a variety of different approaches aimed at mitigating the risks posed by dynamic query construction as well as the wide-ranging implications of these risks for database security will be evaluated in depth.

Zusammenfassung

Injection-Angriffe stellen seit dem Aufkommen von Datenbanksystemen eine dauerhafte und allgegenwärtige Gefahr für diese dar. Während zumeist klassische relationale Datenbanken und SQL attackiert werden, wurde das Angriffssparadigma der Injection im Laufe der Zeit angepasst, um zudem auch NoSQL-Datenbanken und deren Abfragesprachen ins Visier nehmen zu können. Diese Arbeit untersucht das Thema der Cypher-Injection, also das Konzept der klassischen SQL-Injection, angepasst auf die Graphdatenbank Neo4j und deren zugehörige Sprache Cypher. Bestehende Taxonomien für SQL-Injection-Angriffe (SQLIA) werden betrachtet, um die Übertragbarkeit verschiedener Arten von SQLIAs auf Neo4j und Cypher und dafür notwendige Anpassungen zu beurteilen. Zu diesem Zweck untersucht die Arbeit einige zwar häufig verwendete, jedoch nicht eindeutig definierte Begriffe im Zusammenhang mit der Konstruktion und Ausführung von Datenbankabfragen näher, wie etwa *statisch*, *dynamisch* und *parameterisiert*. Dynamischen Abfragen wird hierbei besondere Aufmerksamkeit gewidmet, da diese im Hinblick auf Injection-Angriffe die problematischste Klasse von Abfragen darstellen. Daher wird eine Vielzahl unterschiedlicher Ansätze zur Risikoverminderung in Bezug auf dynamische Konstruktion von Abfragen und die weitreichenden Implikationen dieser Risiken für die Datenbanksicherheit ausführlich untersucht.

Contents

1	Introduction	1
1.1	Objective and Research Question	1
1.2	Structure of the Thesis	1
2	Background	2
2.1	SQL Injection: Introduction and Taxonomy	2
2.2	Query Construction Paradigms	3
2.3	NoSQL Databases and Security	4
2.4	Graph Databases, Neo4j, and Cypher	4
3	Transferring Injection Attacks to Cypher	5
3.1	SQL vs. Cypher Injection	5
3.2	Parameterizable and Non-Parameterizable Queries	6
3.3	Illustrative Injection Attacks	7
3.4	Implications and Existing Defenses	8
4	Countermeasures: Analysis and Evaluation	9
4.1	Classic Parameterization and APOC	9
4.2	Dynamic Labels, Types, and Properties	9
4.3	Data Model Refactoring	10
4.4	Input Validation and Sanitization	10
4.5	Access Control and Least Privilege	11
4.6	Comparative Evaluation	12
5	Conclusion	13
5.1	Summary	13
5.2	Future Work	13
	References	14

1 Introduction

Injection attacks remain a very prominent and frequent security issue, ranking fifth in the 2025 edition of the OWASP Top 10 list [OWA25], which aims to identify the most critical security risks to web applications each year. *Injection* also exhibits the most Common Vulnerabilities and Exposures (CVEs) of all ten categories with a total of 62,445 CVEs, almost double that of *Broken Access Control*, which has the second highest number of CVEs at 32,654 (cf. [OWA25]).

1.1 Objective and Research Question

While query injection techniques and remedies pertaining to SQL and relational databases have been well researched, injection attacks targeting NoSQL databases and their query languages have received much less attention as of yet. According to Van Landuyt et al. [VWJ24], their study represents the first detailed examination of query injections targeting graph databases in particular (cf. [VWJ24], p. 2). Building on their work, this thesis takes on the form of a literature-based analysis and uses the example of *Neo4j*, the most widely used graph database, along with the associated *Cypher* query language. Of particular importance is a recent update to the Neo4j ecosystem which has significantly improved the security of dynamic queries via new parameterization features (cf. [LAM24]). While Van Landuyt et al. [VWJ24] demonstrate the vulnerability of dynamic queries to injection attacks in Cypher, the application of effective countermeasures is left to future research. Following up on the second research question Van Landuyt et al. [VWJ24] examine, namely “Which types of injection attacks can be implemented in the Cypher language when the aforementioned query parameterization approach cannot be enforced?” (see [VWJ24], p. 3), this thesis aims to answer the following research question:

RQ Which countermeasures can be applied to secure dynamically constructed Cypher queries in Neo4j against injection attacks, especially where parameterization is not possible?

A secondary objective of this thesis is the discussion of a number of very commonly used terms related to query construction and execution which are most often taken for granted in academic works on the subject of databases. Due to the in part elusive and loosely defined nature of some of these notions, particular care will be given to properly convey the general idea of *static*, *dynamic*, and *parameterized* queries to the reader.

1.2 Structure of the Thesis

Subsequent to the brief introduction to the main topic of this thesis given in Chapter 1, the general problem of injection attacks will be elaborated using the example of SQL in Chapter 2. This is followed by a discussion of different query construction paradigms and the emergence of NoSQL databases, eventually leading to graph databases using the example of Neo4j and Cypher. Based on this foundation, Chapter 3 transfers common types of injection attacks from SQL to Cypher and offers an in-depth examination of the different kinds of dynamic queries and their individual vulnerabilities and limitations. Additionally, the implications for industry applications and limitations of existing defenses are considered. In Chapter 4, a number of countermeasures aimed at securing dynamic queries against injection attacks are evaluated and compared based on several criteria. The thesis is concluded by Chapter 5 in which findings are summarized and open questions requiring further research are discussed.

2 Background

Here, first a taxonomy for SQL injections and a structured overview of query construction methods are introduced in turn. Afterwards, the topic of security in the broader NoSQL context is examined, eventually leading to graph databases using the example of Neo4j.

2.1 SQL Injection: Introduction and Taxonomy

In an SQL Injection Attack (SQLIA), the attacker injects malicious code or particular SQL statements into a vulnerable application by exploiting the way user input and database access are being carried out. The root cause of such vulnerabilities is well understood and usually mitigated via input validation, defensive coding, and other techniques. However, considering the vast array of different types of SQLIAs and accounting for human error in the application of the mitigation techniques mentioned above, a much different picture of the ensuing threat emerges (cf. [HVO06], p. 53). The following taxonomy is adapted from Halfond et al. [HVO06] and Van Landuyt et al. [VWJ24], and distinguishes between seven distinct major types of SQLIAs summarized in Table 2.1. While Halfond et al. [HVO06] also review a wide range of techniques for detection and prevention of SQLIAs, an in-depth analysis of countermeasures applicable to graph databases and their query languages is deferred to Chapter 4.

Table 2.1: Taxonomy of SQL Injection Attacks, adapted from [HVO06] and [VWJ24].

Attack Type	Intent	Description
Tautology	Auth bypass, injectable parameters, data extraction	Injects code into a query's WHERE clause, so that it is always true.
Illegal/Illogical	DB fingerprinting, injectable parameters, data extraction	Triggers detailed error messages to leak database metadata.
Union Query	Auth bypass, data extraction	Injects a UNION SELECT statement to retrieve data from other tables.
Piggy-Backed	Data modification & extraction, DoS, remote execution	Injects additional statements separated by ; query delimiters.
Stored Procedure	Privilege escalation, DoS, remote execution	Exploits built-in or user-defined stored procedures.
Inference	Schema discovery, injectable parameters, data extraction	Analyzes application response behavior, blind or timing-based.
Alternate Encoding	Detection evasion	Obfuscates the payload using hexadecimal, ASCII, Unicode, etc.

Listing 2.1 serves as an example of application-side code that receives user input and dynamically (i.e. at run time) builds from it a query using string concatenation.

```
1 String query = "SELECT * FROM users WHERE username = '" + login
2               + "' AND password = '" + secret + "';";
```

Listing 2.1: Vulnerable application code that builds an SQL query at run time.

In Listing 2.2, a tautology attack is carried out on the code from Listing 2.1. The username field receives ' OR 1=1 -- as input, causing the compromised query to return all users regardless of credentials. The single-line comment, indicated by -- in SQL, neutralizes the password check, bypassing authentication entirely when the query is executed on the underlying database.

```
1 SELECT * FROM users WHERE username = ' OR 1=1 --' AND password = '';
```

Listing 2.2: Resulting SQL query with ' OR 1=1 -- as username input.

The SQL Injection example illustrated here serves as a baseline for familiarizing the reader with the basic notion of an injection attack and is essential for transitioning to the related concept of Cypher Injection in Chapter 3. Analogous examples for all other attack types listed in Table 2.1 are provided by Halfond et al.'s study (cf. [HVO06], pp. 55–57).

2.2 Query Construction Paradigms

Van Landuyt et al. [VWJ24] distinguish between *static*, *parameterized* queries whose structure is already known at compile time and *dynamic* queries constructed at run time via string concatenation (cf. [VWJ24], p. 3). Due to the fact that the meaning of these terms remains ambiguous and can vary depending on the context, an expanded classification is introduced in Table 2.2 for the purpose of this thesis, indicating vulnerability to injection attacks for each construction paradigm. Note that *Static* and *Parameterized* are treated as a single category by Van Landuyt et al. [VWJ24], since parameterized queries are just as secure as static queries in the context of injection attacks – provided that the underlying parameter passing mechanism is implemented soundly. The distinction between *Mixed* and *Dynamic* serves merely to illustrate the possibility of mixing different query construction paradigms freely, and also highlights the fact that a query that is only partially parameterized remains susceptible to injection.

Table 2.2: Spectrum of query construction paradigms.

Construction	Description	SQL Example	Safe?
Static	Query structure is fixed and fully known at compile time.	"SELECT * FROM users;"	Yes
Parameterized	Query structure is fixed at compile time, parameters are passed separately at run time.	"SELECT * FROM users WHERE id = ?;"	Yes
Mixed	Query structure is assembled at run time, yet also includes parameters, query as whole is dynamic.	"SELECT * FROM users WHERE id = ? ORDER BY ' + col + "' ASC;"	No
Dynamic	Query structure is assembled at run time, has no parameterized elements.	"SELECT * FROM " + table + " WHERE " + cond + " = '" + val + "';"	No

Although Listing 2.1 technically constitutes an example of dynamic query construction, it can easily be parameterized and thereby fully secured against query injection – this will be further elaborated in Chapter 3 and Chapter 4. Generally speaking, the basic concepts regarding query construction and injection attacks which are outlined here using the example of SQL are also applicable to Cypher, barring some technicalities and transfer problems.

2.3 NoSQL Databases and Security

While the term NoSQL was originally associated with databases that did not support the traditional SQL interface (cf. [SRC22], p. 1), it now more commonly refers to a number of different database technologies and query languages departing from the relational model (cf. [HQL16], p. 75). Modern NoSQL databases can be categorized based on their underlying data model, of which there are four different predominant types: *key-value*, *column-oriented*, *document*, and *graph*. One major difference between relational and NoSQL databases is that, while the former make use of a static schema suitable only for structured data, the latter are able to handle structured, semi-structured, and unstructured data (cf. [SRC22], p. 5). This can be attributed to the schema-less or schema-flexible nature of NoSQL databases (cf. [GAG18], p. 109). While it is standard for relational databases to adhere to the *ACID* paradigm (i.e. Atomicity, Consistency, Isolation, Durability), many NoSQL variants cannot guarantee these features and stick to the less strict *BASE* paradigm (i.e. Basically Available, Soft State, Eventual Consistency) instead (cf. [SRC22], p. 6). Nonetheless, some NoSQL databases (such as Neo4j) are fully *ACID*-compliant with regard to transactions (cf. [GAG18], p. 116). In terms of security, NoSQL technology turns out to be rather problematic. This is owed to the wide range of different data models and design paradigms, and also to the fact that many NoSQL databases were not originally conceived with appropriate security mechanisms in mind. Unlike relational databases, they frequently lack essential security features, particularly with regard to file encryption, access control, and user authentication (cf. [GAG18], pp. 102–103). The comparative study conducted by Fahd et al. [FVH19] comes to the conclusion that many highly popular NoSQL databases offer only very basic security features (cf. [FVH19], pp. 4–8).

2.4 Graph Databases, Neo4j, and Cypher

While relational databases are based on the relational model, graph databases are based on the property graph model. In this data model, information is not stored in tables, but in a graph structure consisting of nodes and edges. Semantically, nodes are meant to represent entities in the real world, whereas edges model the entities' relationships to each other. In the property graph model, nodes can have any number of labels and properties, representing the roles and characteristics of each node respectively. Relationships can also have an arbitrary number of properties, but in any case must have exactly one source node, one target node, and one relationship type (cf. [GGL19], p. 2243). The most prominent example of such a graph database is *Neo4j* along with its query language *Cypher* (cf. [VWJ24], p. 2). While *Cypher* shares a number of similarities with SQL with regard to its design, there are numerous differences in terms of how freely statements can be constructed (cf. [GGL19], p. 2242). Unlike SQL, *Cypher* is characterized by its particular "ASCII art"-style syntax, which aims to enable the user to visually define querying patterns with respect to the underlying graph (cf. [BMU17], p. 304). Regarding the topic of security, Fahd et al. [FVH19] assess the security features of *Neo4j* to be mostly up to par with or in some cases even better than several other NoSQL databases it is compared against – scoring a security level of *Medium* regarding built-in security configuration, *Low-Medium* for encryption, *Medium* in the case of authentication and authorization, and *Medium* for auditing features (cf. [FVH19], pp. 4–8). It must be noted that, as indicated in the *Neo4j* documentation, the scope of security features varies notably depending on whether the commercial *Enterprise Edition* or the free *Community Edition* is used (cf. [NIN26]). One feature available only in the commercial version is the support of role-based security, a crucial countermeasure against injection attacks which will be examined more closely in Chapter 4.

3 Transferring Injection Attacks to Cypher

In this chapter, first a comparison is drawn between injection attacks on SQL and Cypher queries. Then, a classification scheme for dynamic queries is introduced, followed by a number of application and attack code examples. Finally, the implications of Cypher Injection for common industry applications and the limitations of existing defenses are considered.

3.1 SQL vs. Cypher Injection

In the Neo4j knowledge base, Cypher Injection is described as *"a way for maliciously formatted input to jump out of its context, and by altering the query itself, hijack the query and perform unexpected operations on the database."* (see [BLA23]). This closely mirrors conventional definitions of SQL Injection, such as the one supplied by Halfond et al. (cf. [HVO06], p. 53). As demonstrated by Van Landuyt et al. [VWJ24], the Cypher language is susceptible to a wide array of injection attack types which were directly adapted from existing SQLIA taxonomies (cf. [VWJ24], p. 2), reinforcing the notion that injection attacks take on a similar form in the context of both relational and NoSQL databases. Based on this observation, Table 3.1 maps the SQLIA taxonomy established in Table 2.1 to the Cypher query language, indicating for each attack type whether it requires Cypher-specific adaptation or falls outside the scope of this thesis. This is achieved by comparing the SQL Injection examples provided by Halfond et al. [HVO06] with the Cypher Injection examples demonstrated by Van Landuyt et al. [VWJ24].

Table 3.1: Mapping of the SQLIA taxonomy (Table 2.1) to Cypher.

Attack Type	Applicability	Realization in Cypher
Tautology	requires adaptation	Inject into the WHERE clause using // instead of --, property matching queries require further structural adaptation.
Illegal/Illogical	requires adaptation	Cypher error messages reveal different metadata than SQL, no schema or tables.
Union Query	requires adaptation	Inject UNION between MATCH clauses, no SELECT equivalent in Cypher.
Piggy-Backed	requires adaptation	Close open structures and/or use CALL subqueries, no query delimiter chaining via ; as in SQL.
Stored Procedure	requires adaptation	User-defined functions need to be tailored to Neo4j (cf. [VWJ24], Listing 6).
Inference	not covered by [VWJ24]	Outside the scope of this thesis, discussed in Chapter 5 as future work.
Alternate Encoding	not covered by [VWJ24]	Outside the scope of this thesis, discussed in Chapter 5 as future work.

As can be seen in Table 3.1, while all five SQLIA types covered by Van Landuyt et al. [VWJ24] have conceptual counterparts in the Cypher language, none of them can be transferred directly

without substantial Cypher-specific adaptation, examples of which can be seen later in this chapter. These modifications may concern syntax and query structure, the underlying data model, deployment configurations specific to Neo4j, or possibly any combination of the above. Note that the feasibility of *Inference* and *Alternate Encoding* attacks in the Neo4j ecosystem was neither discussed nor demonstrated by Van Landuyt et al. [VWJ24]. While this lies outside the scope of this thesis, it will be briefly discussed in Chapter 5 as future work.

3.2 Parameterizable and Non-Parameterizable Queries

In this thesis, dynamic queries which can be fully parameterized are called *parameterizable*, whereas dynamic queries which cannot be (fully) parameterized, and therefore require run-time construction mechanisms such as string concatenation, are called *non-parameterizable*. Listing 3.1 presents an example of a parameterizable dynamic query in the Cypher language which is unnecessarily constructed at run time via string concatenation. When transformed into an equivalent parameterized query, it moves from the category of *Dynamic* to *Parameterized* as per Table 2.2. The securing of dynamic queries via parameterization as well as by other means in cases where parameterization is not possible will be discussed at length in Chapter 4.

```
1 String query = "MATCH (u:User) WHERE u.username = '" + login
2               + "' AND u.password = '" + secret + "' RETURN u";
```

Listing 3.1: Application code that concatenates string values at run time.

The key factor in determining whether a construct or clause can be parameterized is its structural role in the query. Whereas simple values like literals and expressions as well as node and relationship IDs can easily be parameterized in Cypher, the same cannot be said with regard to elements that directly make up the query structure (cf. [NPA26]). While node labels, relationship types, and property names are traditional examples of such non-parameterizable structural elements that are vulnerable to injection attacks (cf. [BLA23]), recent updates to the Cypher language have introduced notable new security features in this regard. This turns out to be crucial in the case of Listing 3.2 – while the `:User` node label is already fixed at compile time and therefore not susceptible to injection, the node label representing the account is determined only at run time and thus can be injected into. Unlike the query shown in Listing 3.1, this second dynamic query was not parameterizable in the past, but this has changed recently.

```
1 String query = "MATCH (u:User)-[:CAN_ACCESS]->(a:'" + account + "') RETURN u";
```

Listing 3.2: Application code that concatenates a node label at run time.

Starting from Neo4j 5.26, the current Long-Term Support release, the use of dynamic labels, types, and properties is supported (cf. [MCC25]) – a feature further explained in Chapter 4. This turns out to be a notable shift in the attack surface for Cypher Injection, since a number of previously non-parameterizable dynamic queries such as Listing 3.2 can now be fully parameterized provided a number of caveats are taken into account (cf. [LAM24]). Listing 3.3 and Listing 3.4, on the other hand, exemplify the more problematic notion of a truly non-parameterizable dynamic query as of now. Since structural keywords remain unable to be parameterized, the fact that the sort direction keyword (either explicitly `ASC` or `DESC`, otherwise defaulting to `ASC`) (cf. [NOR26]) is decided only at run time in Listing 3.3 dictates that the entire query must be built dynamically and is therefore vulnerable to injection attacks.

```
1 String query = "MATCH (u:User) RETURN u ORDER BY u.name " + dir;
```

Listing 3.3: Application code that concatenates a sort direction at run time.

Analogously, owed to the fact that operators are not parameterizable either, the comparison operator (either =, <>, <, >, <=, >=, IS NULL, or IS NOT NULL) (cf. [NCO26]) concatenated at run time in Listing 3.4 necessitates dynamic query construction. Importantly, it must be noted that sort directions and comparison operators are not an exhaustive list but merely two possible examples of structural elements that remain non-parameterizable, just as before.

```
1 String query = "MATCH (u:User) WHERE u.age " + op + " 23 RETURN u";
```

Listing 3.4: Application code that concatenates a comparison operator at run time.

3.3 Illustrative Injection Attacks

To provide a better understanding of Cypher Injection in practice, a number of injection attacks of different types targeting the previously introduced query examples are presented below. For lack of a practical demonstration in a test environment, the realization of each attack is argued and justified only in theory. While differing in form and structure, the technical feasibility of the attack types used in this section has been demonstrated by Van Landuyt et al. [VWJ24].

The tautology attack illustrated in Listing 3.5 targets the query constructed by Listing 3.1. In contrast to its SQL equivalent (Listing 2.2), the payload is ' OR 1=1 RETURN u // rather than ' OR 1=1 --. This is owed to two differences from SQL: single-line comments are indicated by // rather than --, and read queries must contain a RETURN clause to yield a result, forcing the attacker to reintroduce a RETURN u clause, since the original one is neutralized by the comment.

```
1 MATCH (u:User) WHERE u.username = '' OR 1=1 RETURN u //' AND u.password = ''
   RETURN u
```

Listing 3.5: Resulting Cypher query with malicious username input payload.

Listing 3.6 illustrates a more elaborate piggy-backed injection attack which aims to perform data extraction by supplying acc013')-[:CAN_ACCESS]->(s:Secret) RETURN s // as a payload to the application in Listing 3.2. By closing the open node label structure and appending additional Cypher statements, the attacker would be able to return not just users with access to a certain account (here: acc013) as arranged by the query template, but to retrieve all :Secret nodes instead, here used to represent confidential information, which this account can access.

```
1 MATCH (u:User)-[:CAN_ACCESS]->(a:'acc013')-[:CAN_ACCESS]->(s:Secret) RETURN s
   //' RETURN u
```

Listing 3.6: Resulting Cypher query with malicious node label payload.

An example of a union query attack on a truly non-parameterizable query (see Listing 3.3) is provided in Listing 3.7. As described in Table 3.1, this can be achieved by combining the result sets of two MATCH clauses via the UNION keyword. For this purpose, the attacker injects the payload ASC UNION MATCH (u:Invoice) RETURN u into the sort direction field of the query, effectively retrieving not just all of the users but also all :Invoice nodes from the database.

```
1 MATCH (u:User) RETURN u ORDER BY u.name ASC UNION MATCH (u:Invoice) RETURN u
```

Listing 3.7: Resulting Cypher query with malicious sort direction payload.

Finally, another tautology attack on a second non-parameterizable query (see Listing 3.4) is outlined in Listing 3.8. Here, the payload `>= 0 OR 1=1 RETURN u //` is sent to the application and concatenated to the comparison operator field of the query. Instead of merely providing the operator and returning nodes for which the inequality holds as designated by the query template, the attacker is able to retrieve all user nodes regardless of age from the database.

```
1 MATCH (u:User) WHERE u.age >= 0 OR 1=1 RETURN u // 23 RETURN u
```

Listing 3.8: Resulting Cypher query with malicious comparison operator payload.

3.4 Implications and Existing Defenses

Van Landuyt et al. [VWJ24] argue that exploiting the full potential of graph databases may require enabling the user to interactively query and traverse the underlying graph at run time – a function which naturally lends itself to the paradigm of dynamic query construction. They identify dynamic queries to be indispensable for a myriad of graph database use cases such as fraud detection, recommendation systems, financial services, government, social networks, as well as artificial intelligence and analytics (cf. [VWJ24], pp. 7–8). Evidently, the real-world application of graph database technology spans across a wide range of industries, being suitable for both the private and public sectors. In the context of this thesis, the crucial finding is that all of the use cases listed above necessarily require the use of dynamic query construction in practice and therefore exhibit inherent vulnerability to Cypher Injection. While this assessment remains largely unscathed by the recent introduction of dynamic labels, types, and properties, its implications and immediate risks for database security may turn out to be less dire than they might have been at the time Van Landuyt et al.’s [VWJ24] study was written.

While dynamic query construction is generally discouraged in the Neo4j knowledge base, the findings of Van Landuyt et al. [VWJ24] have shown that there are many concrete use cases, such as the ones mentioned above, for which it cannot be avoided. The primary mechanism for building dynamic queries continues to be string concatenation, which is inherently vulnerable to injection attacks. This is owed to the fact that Neo4j does not support safer alternative mechanisms like query factories for dynamic query construction (cf. [VWJ24], p. 3). This represents a major problem, since queries in which non-parameterizable elements are to be determined at run time cannot be fully parameterized. Proposed remedies include the use of input validation, sanitization, role-based access control, and the Awesome Procedures On Cypher (APOC) library. While these approaches are considered best practice in cases where string concatenation is indispensable, they are prone to human error and inferior to parameterization in most regards (cf. [BLA23]). However, it must be acknowledged that the recent introduction of dynamic labels, types, and properties has turned a large chunk of previously non-parameterizable queries into parameterizable ones, although the complete migration of these queries to their new dynamic form may require extensive efforts (cf. [LAM24]). Regardless, a certain residual class of queries that are still non-parameterizable (see Listing 3.3 and Listing 3.4) remains. While further research is required to fully mitigate these, several countermeasures that offer at least partial security will be discussed in Chapter 4.

4 Countermeasures: Analysis and Evaluation

This chapter examines a number of approaches to secure dynamic queries, ranging from techniques based on parameterization and refactoring to others like query sanitization and access control. Whereas many dynamic queries can easily be fully secured via parameterization, there is no proper mechanism to secure non-parameterizable queries equally well as of now.

4.1 Classic Parameterization and APOC

The consistent use of parameterized queries constitutes a strong defense against the threat posed by Cypher Injection. Van Landuyt et al. [VWJ24] have thoroughly demonstrated the strict separation of query specification and parameter passing in the Neo4j ecosystem, concluding that the residual injection risks are negligible when the built-in query parameterization mechanism is employed (cf. [VWJ24], pp. 3–6). To secure the application code from Listing 3.1 against an injection attack such as the one presented in Listing 3.5, it can be parameterized as shown in Listing 4.1. The parameters `$login` and `$secret` are here not actually included in the string, but rather passed to the database client securely and evaluated separately.

```
1 String query = "MATCH (u:User) WHERE u.username = $login AND u.password =  
    $secret RETURN u";
```

Listing 4.1: Secure code using two parameters for property values.

Frequently used in conjunction with parameterization is the APOC plugin for Neo4j, a library which offers additional functionality that also enables working with run-time data more effectively (cf. [LAM24]). APOC is divided into a core and an extended module, which are made up of over 500 widely used procedures and functions in total (cf. [NAP26]). While parameterization represents the optimal way to secure dynamic queries, it is only applicable to queries that are *parameterizable* as defined in Chapter 3. For *non-parameterizable* queries there exist a number of best practices and pragmatic approaches discussed in the following sections.

4.2 Dynamic Labels, Types, and Properties

From Neo4j 5.26 onward, parameters can be used to replace node labels, relationship types, and property names in queries where these elements are referenced dynamically (cf. [NPA26]). While the implementation of this feature is not fully optimized for performance yet, it renders the injection vulnerability of applicable queries void. In the Neo4j developer blog, this change is described as a departure from APOC, as it is recommended to migrate old APOC queries to native Cypher by using dynamic labels, types, and properties where possible (cf. [LAM24]). This increases the flexibility of query construction and acts as a defense against injection attacks (cf. [MCC25]). The application code from Listing 3.2 can be fully secured by parameterizing it as shown in Listing 4.2. Here, `$()` denotes the dynamic node label whose value is not hard-coded, but rather determined by the enclosed parameter `$accountNr` at run time.

```
1 String query = "MATCH (u:User)-[:CAN_ACCESS]->(a:$($accountNr)) RETURN u";
```

Listing 4.2: Secure code using a dynamic label with enclosed parameter.

Despite marking a big step forward in solving the problem of Cypher Injection, this novel feature does not come without downsides. The query planner can only optimize queries based on data available at compile time, but queries employing a dynamic label, for instance, consequently have a structural element (here: the node label) which is supplied only at run time. Therefore, the resulting query plan will turn out to be much less efficient than if the node label were already known at compile time. The crucial distinction is that, unlike classic parameterization, the parameter value passed to the database at run time is not just a literal or expression in this case, but rather a structural element of the query. For this reason, the use of dynamic labels, types, and properties is only recommended if an equivalent query cannot possibly be constructed statically or using only classic parameterization (cf. [LAM24]).

4.3 Data Model Refactoring

Since parameterization constitutes the optimal defense against injection attacks, a pragmatic way to avoid having to resort to dynamic queries is to refactor the underlying data model in a way that allows for corresponding queries to be written in parameterized form. While this approach may sound like an ideal solution at first glance, constraints regarding the data model and the required refactoring efforts may render it unfeasible for many applications (cf. [BLA23]). A notable advantage over the use of dynamic labels, types, and properties is that the query planning and execution performance is unaffected. Sticking with the example of Listing 3.2, the underlying data model could be changed to enable writing a semantically equivalent parameterized query instead of using a dynamic label. In Listing 4.3, following the advice given by Bowman and Lamont [BLA23] in the Neo4j knowledge base, the account number is altered from being the node label itself to being a property of all `:Account` nodes.

```
1 String query = "MATCH (u:User)-[:CAN_ACCESS]->(a:Account {nr: $accountNr})  
  RETURN u";
```

Listing 4.3: Secure code using refactored data model and parameterization.

This change would constitute a sensible refactoring regardless of the injection vulnerability and query execution concerns it remedies. Refactoring is recommended in case of poor performance (cf. [NRE26]), which could certainly be the case when each account number is stored as a separate node label. However, it must be noted that refactoring the data model does not genuinely *secure* dynamic queries and instead simply avoids them in the first place.

4.4 Input Validation and Sanitization

In cases where dynamic queries cannot possibly be avoided, query sanitization is commonly used as a pragmatic means of defense against injection attacks. Sanitization generally refers to the purposeful manipulation of user input to guarantee its validity and prevent illegal input. While the exact method used to achieve this goal can vary by database and query language, in Cypher it commonly takes on the form of escaping or filtering out delimiter and closing characters in the user input which would otherwise be interpreted as part of the underlying query. The purpose of the related concept of input validation, on the other hand, is not to clean or modify the user input, but rather to check whether it complies with certain requirements and to accept or reject it based on that evaluation (cf. [BLA23]). This is often achieved by filtering for specific keywords or characters that could indicate harmful or otherwise unusual user input

(cf. [SSA21], p. 3). Naturally, these two concepts complement each other and are often used in conjunction, but possess individual risk profiles nonetheless (cf. [BLA23]). Listing 4.4 shows how to sanitize the query from Listing 3.6 following the method outlined by Bowman and Lamont [BLA23], that is, by simply escaping the maliciously injected extra backtick via another backtick character. As indicated by the escaped payload in Listing 4.4, the sanitized query now treats the attack payload as a simple string to be used as a node label. As such, it poses a much smaller threat since the underlying query structure shown in Listing 3.2 remains unaltered.

```
1 MATCH (u:User)-[:CAN_ACCESS]->(a:'acc013''')-[:CAN_ACCESS]->(s:Secret) RETURN s
   //(') RETURN u
```

Listing 4.4: Sanitized Cypher query with neutralized node label payload.

Islam et al. [IIA19] find that sanitization on its own is unable to fully protect queries and applications against injection, leaving NoSQL databases in particular exposed to a wide range of possible injection attacks. For a more exhaustive mitigation of such attacks they present a piece of software that aims to automatically detect injection attacks in the context of NoSQL databases (cf. [IIA19], pp. 760–761). Regarding validation, Halfond et al. [HVO06] identify two distinct forms the technique can take. Whereas *positive* validation aims to recognize and accept only legal input, *negative* validation does the exact opposite, specifically looking for illegal input to be rejected. While both approaches make use of pattern matching as their core mechanism, positive validation is recommended over its counterpart, as it is inherently more secure to define all forms of acceptable input than to try covering all cases of input that should be rejected (cf. [HVO06], p. 58). Examples where validation rather than just sanitization is sensible are the queries constructed in Listing 3.3 and Listing 3.4, as their dynamic fields take only a certain set of allowed keywords. A possible way to secure these queries could be to construct corresponding allowlists (i.e. positive validation) for the sort direction $\in \{\text{ASC}, \text{DESC}, \emptyset\}$ and the comparison operator $\in \{=, <, >, <=, >=, \text{IS NULL}, \text{IS NOT NULL}\}$ fields respectively.

4.5 Access Control and Least Privilege

Another important defense against injection attacks is the use of access control mechanisms such as role-based access control (RBAC). Putting this access control paradigm into practice entails assigning to each user one or more roles that directly dictate which permissions are granted to them (cf. [SRC22], p. 7). In Neo4j, RBAC is based on a combination of allowlisting and denylisting approaches, and is supported by way of offering a number of built-in roles, namely *reader*, *editor*, *publisher*, *architect*, and *admin*. Each of these roles entails different degrees of database access and associated privileges, and can be mixed freely per user. Neo4j also offers two other methods of access control, namely *sub-graph* and *property-level* access control. While the former is also based on a combined allowlist/denylist mechanism, the latter employs only denylisting to manage or restrict user privileges (cf. [SRC22], p. 11). Listing 4.5 illustrates how read access to the `:Secret` nodes, which are used in this thesis to represent confidential data, can be restricted for a particular role, such as the built-in *reader* role. This is achieved by following the instructions outlined in the Neo4j operations manual (cf. [NRB26]).

```
1 DENY MATCH {*} ON GRAPH neo4j NODES Secret TO 'reader'
```

Listing 4.5: Denying *reader* users access to `:Secret` nodes via RBAC.

An important caveat to keep in mind is that RBAC functionality is only available in the commercial *Enterprise Edition* of Neo4j (cf. [NIN26]). The Neo4j operations manual also stresses the importance of least privilege, a concept which demands that users are only given the exact minimal set of access rights that are absolutely required for their tasks. This constitutes a strong pragmatic mitigation technique against injection attacks as compromised queries are merely able to cause damage limited to the scope of the attacker’s role. In this regard, it is crucial that role-based privileges are managed based on the principle of least privilege in order to reduce the attack surface, for example by making read-only access the default setting and granting write privileges only to selected roles (cf. [NRB26], [BLA23]).

4.6 Comparative Evaluation

The five countermeasures discussed in this chapter differ from each other significantly in their effectiveness, the mechanism they employ, the class of queries they pertain to, as well as their limitations. Table 4.1 draws a comparison between these, indicating per countermeasure whether its principal mechanism actually *prevents* an injection attack, merely *contains* its impact once it has already occurred, or simply *avoids* dynamic queries altogether. Additionally, it is evaluated whether the respective countermeasure applies to non-parameterizable dynamic queries in the strict sense of Chapter 3, that is, queries whose unknown element at compile time directly makes up the query structure. Dynamic labels, types, and properties are here recognized as *partially* securing such queries, albeit in a different way than input validation and sanitization. Whereas the former has effectively moved a large chunk of *non-parameterizable* queries to the category of *parameterizable*, the latter is much more cumbersome and imperfect, but conversely is also able to remedy the queries that remain in the *non-parameterizable* class.

Table 4.1: Comparative evaluation of examined countermeasures.

Countermeasure	Mechanism	Secures non-para. dynamic queries?	Limitations
Classic Parameterization / APOC	prevention	no	Covers only literals, expressions, node/relationship IDs, works only for regular parameterizable queries.
Dynamic Labels, Types, Properties	prevention	partially	Covers only labels, types, properties, not other structural elements, worsens query planning and execution.
Data Model Refactoring	avoidance	no	Merely avoids dynamic queries, data model constraints, extensive refactoring efforts.
Input Validation / Sanitization	prevention	partially	Prone to human error, time-consuming, allowlisting only feasible if values are known.
Access Control / Least Privilege	containment	no	Does not prevent injection, requires commercial edition, prone to read-based attacks.

5 Conclusion

This thesis presented a detailed overview of the problems posed by injection attacks in the context of graph databases and their query languages. At the core of this investigation stood the question of how to secure dynamic queries written in the Cypher language against injection attacks, with particular focus on cases where classic parameterization was not supported.

5.1 Summary

After a brief introduction to the topic provided by Chapter 1, the ever-present problem of injection attacks was explained using SQL in Chapter 2, laying a foundation for later chapters. This was followed by a discussion of different query construction paradigms and the emergence of NoSQL databases, covering graph databases and their query languages using the example of Neo4j and Cypher. Chapter 3 then transferred the previously established SQLIA taxonomy to Cypher and classified different kinds of dynamic queries based on their ability to be parameterized. Finally, the real-world implications for contemporary applications of graph database technology and the current state of defenses against Cypher Injection were studied, leading to Chapter 4. Here, various frequently used countermeasures for mitigating the dangers posed by injection on dynamic queries were evaluated. In particular, this included different approaches using parameterization, dynamic elements, refactoring, validation, sanitization, and access control. While each one of these countermeasures can aid significantly in mitigating the numerous kinds of injection attacks, it must be noted that, at the time of writing, there exists no all-encompassing solution for securing dynamic queries. Nonetheless, the introduction of dynamic labels, types, and properties marks a major step towards achieving this final goal.

5.2 Future Work

Initially, it must be noted that a practical demonstration of the feasibility of *Inference* and *Alternate Encoding* injection attacks using Cypher is still outstanding, as this was omitted in Van Landuyt et al.’s [VWJ24] study. Also, further surveying is needed to possibly discover yet unheard-of types of injection attacks pertaining to graph databases, or Neo4j and Cypher in particular. The possibility of developing fully automated tools for detecting conventional SQL injections has been demonstrated in the past by Wassermann and Su [WSU07], for instance. More recently, machine learning solutions for detecting NoSQL injections across different database paradigms, such as the models proposed by Islam et al. [IIA19], Mejia-Cabrera et al. [MPV21], and Praveen et al. [PDM22], constitute another promising line of defense against injection attacks. This future direction is reinforced by the work of Alghawazi et al. [AAA22], which provides a detailed review of the many papers dealing with detection of injection attacks via machine learning. Seth et al. [SBE25] have thoroughly evaluated the efficacy of Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP), concluding that IAST tools especially are highly recommendable for injection vulnerability detection. While RASP solutions cannot serve as a substitute for the functionality covered by IAST, they represent a useful secondary measure by protecting applications against active injections at run time (cf. [SBE25], p. 1). While these tools have not yet been evaluated in the context of graph databases or the Neo4j ecosystem at the time of writing, Van Landuyt et al. [VWJ24] identify them as highly promising for this purpose (cf. [VWJ24], p. 7).

References

- [AAA22] M. Alghawazi, D. Alghazzawi and S. Alarifi. 'Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review'. In: *Journal of Cybersecurity and Privacy* 2.4 (2022), pp. 764–777. doi: 10.3390/jcp2040039.
- [BLA23] A. Bowman and G. Lamont. *Protecting against Cypher Injection*. URL: <https://neo4j.com/developer/kb/protecting-against-cypher-injection/> (visited on 29th July 2026).
- [BMU17] M. H. Boza and A. Muñoz. '(In) Security in Graph Databases - Analysis and Data Leaks.' In: *SECRYPT*. 2017, pp. 303–310.
- [FVH19] K. Fahd, S. Venkatraman and F. K. Hammeed. 'A Comparative Study of NoSQL System Vulnerabilities with Big Data'. In: *International Journal of Managing Information Technology* 11.4 (2019), pp. 1–19.
- [GAG18] N. Gupta and R. Agrawal. 'NoSQL Security'. In: *Advances in Computers*. Vol. 109. Elsevier, 2018, pp. 101–132.
- [GGL19] A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Schuster, P. Selmer and H. Voigt. 'Updating Graph Databases with Cypher'. In: *45th International Conference on Very Large Data Bases (VLDB)*. Vol. 12. 2019, pp. 2242–2254. doi: 10.14778/3352063.3352139.
- [HQL16] B. Hou, K. Qian, L. Li, Y. Shi, L. Tao and J. Liu. 'MongoDB NoSQL Injection Analysis and Detection'. In: *2016 IEEE 3rd International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE. 2016, pp. 75–78. doi: 10.1109/CSCloud.2016.57.
- [HVO06] W. G. J. Halfond, J. Viegas and A. Orso. 'A Classification of SQL Injection Attacks and Countermeasures'. In: *Proceedings of the International Symposium on Signals, Systems, and Electronics (ISSSE)*. 2006, pp. 53–63.
- [IIA19] M. R. U. Islam, M. S. Islam, Z. Ahmed, A. Iqbal and R. Shahriyar. 'Automatic Detection of NoSQL Injection Using Supervised Learning'. In: *Proceedings of the 43rd Annual Computer Software and Applications Conference (COMPSAC)*. Vol. 1. IEEE. 2019, pp. 760–769. doi: 10.1109/COMPSAC.2019.00113.
- [LAM24] G. Lamont. *Cypher dynamism: A step toward simpler and more secure queries*. 20th Dec. 2024. URL: <https://neo4j.com/blog/developer/cypher-dynamism/> (visited on 29th July 2026).
- [MCC25] I. McCloy. *Neo4j v5 long-term support and the continued evolution*. 17th Jan. 2025. URL: <https://neo4j.com/blog/developer/neo4j-v5-lts-evolution/> (visited on 29th July 2026).
- [MPV21] H. I. Mejia-Cabrera, D. Paico-Chileno, J. H. Valdera-Contreras, V. A. Tuesta-Montez and M. G. Forero. 'Automatic Detection of Injection Attacks by Machine Learning in NoSQL Databases'. In: *Mexican Conference on Pattern Recognition*. Springer. 2021, pp. 23–32. doi: 10.1007/978-3-030-77004-4_3.

REFERENCES

- [NAP26] Neo4j, Inc. *Awesome Procedures On Cypher (APOC)*. URL: <https://neo4j.com/labs/apoc/> (visited on 29th July 2026).
- [NCO26] Neo4j, Inc. *Comparison operators*. URL: <https://neo4j.com/docs/cypher-manual/current/expressions/predicates/comparison-operators/> (visited on 29th July 2026).
- [NIN26] Neo4j, Inc. *Cypher and Neo4j*. URL: <https://www.neo4j.com/docs/cypher-manual/current/introduction/cypher-neo4j/> (visited on 29th July 2026).
- [NOR26] Neo4j, Inc. *ORDER BY: Ascending and descending order*. URL: <https://neo4j.com/docs/cypher-manual/current/clauses/order-by/#ascending-descending-order> (visited on 29th July 2026).
- [NPA26] Neo4j, Inc. *Parameters*. URL: <https://www.neo4j.com/docs/cypher-manual/current/syntax/parameters/> (visited on 29th July 2026).
- [NRB26] Neo4j, Inc. *Role-based access control*. URL: <https://neo4j.com/docs/operations-manual/current/authentication-authorization/manage-privileges/> (visited on 29th July 2026).
- [NRE26] Neo4j, Inc. *Tutorial: Refactor a graph data model*. URL: <https://neo4j.com/docs/getting-started/data-modeling/tutorial-refactoring/> (visited on 29th July 2026).
- [OWA25] OWASP Foundation, Inc. *OWASP Top 10:2025*. URL: <https://owasp.org/Top10/2025/> (visited on 29th July 2026).
- [PDM22] S. Praveen, A. Dcouth and A. Mahesh. 'NoSQL Injection Detection Using Supervised Text Classification'. In: *2022 2nd International Conference on Intelligent Technologies (CONIT)*. IEEE. 2022, pp. 1–5. doi: 10.1109/CONIT55038.2022.9848017.
- [SBE25] A. Seth, S. Bhattacharya, S. Elder, N. Zahan and L. Williams. 'Comparing effectiveness and efficiency of Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP) tools in a large java-based system'. In: *Empirical Software Engineering* 30.3 (2025), pp. 1–27. doi: 10.1007/s10664-025-10621-5.
- [SRC22] S. Sicari, A. Rizzardi and A. Coen-Porisini. 'Security & privacy issues and challenges in NoSQL databases'. In: *Computer Networks* 206 (2022), pp. 1–14. doi: 10.1016/j.comnet.2022.108828.
- [SSA21] M. Shachi, N. S. Shourav, A. S. Ahmed, A. A. Brishty and N. Sakib. 'A Survey on Detection and Prevention of SQL and NoSQL Injection Attack on Server-side Applications'. In: *International Journal of Computer Applications* 183.10 (2021), pp. 1–7.
- [VW]24] D. Van Landuyt, V. Wijshoff and W. Joosen. 'A study of NoSQL query injection in Neo4j'. In: *Computers & Security* 137 (2024), pp. 1–9. doi: 10.1016/j.cose.2023.103590.
- [WSU07] G. Wassermann and Z. Su. 'Sound and Precise Analysis of Web Applications for Injection Vulnerabilities'. In: *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2007, pp. 32–41. doi: 10.1145/1250734.1250739.